

Virtual Machines !!

Virtualization means **creating a virtual** version of something **instead of using the real**, physical one. A physical computer can be divided into smaller parts, and each part can act as its own computer. These separate virtual systems are called **virtual machines (VMs)**.

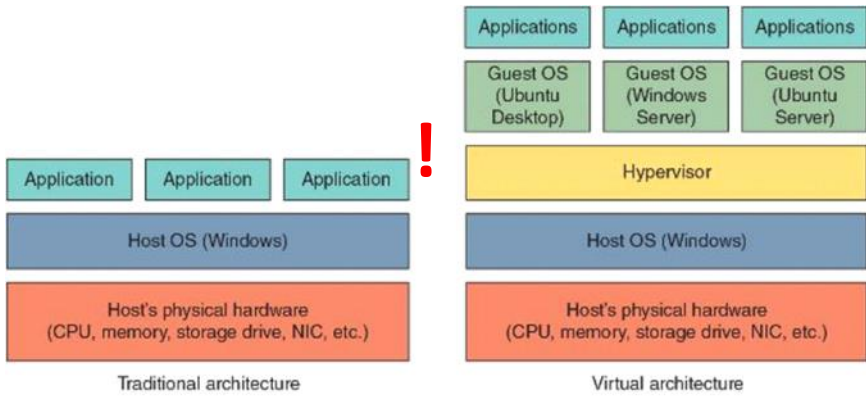


Figure 2-2 A hypervisor allows for installation of multiple guest VMs

- In a **traditional computer**, you have:
 - Physical hardware (CPU, memory, storage)
 - One operating system
 - Applications running on it
- In a **virtual machine setup**, you have:
 - Physical hardware
 - A host operating system
 - A **hypervisor** (special software)
 - Many **virtual machines**
 - Each VM can have its own operating system (like Windows or Linux)

Hypervisors !!

There are two main types of **hypervisors** used for virtualization:

- A **Type 1 hypervisor** is installed **directly on the computer's hardware** (not on an operating system). It is also called a **bare-metal hypervisor**. It is **faster, stronger, and more secure**.
- A **Type 2 hypervisor** is installed on top of an existing operating system (like Windows or Linux) and **works like an application**.

For virtualization to work, the **CPU must support hardware-assisted virtualization (HAV)**.

- On **Intel** processors, this feature is called **VT (Virtualization Technology)**.
- On **AMD** processors, it is called **AMD-V (AMD Virtualization)**.

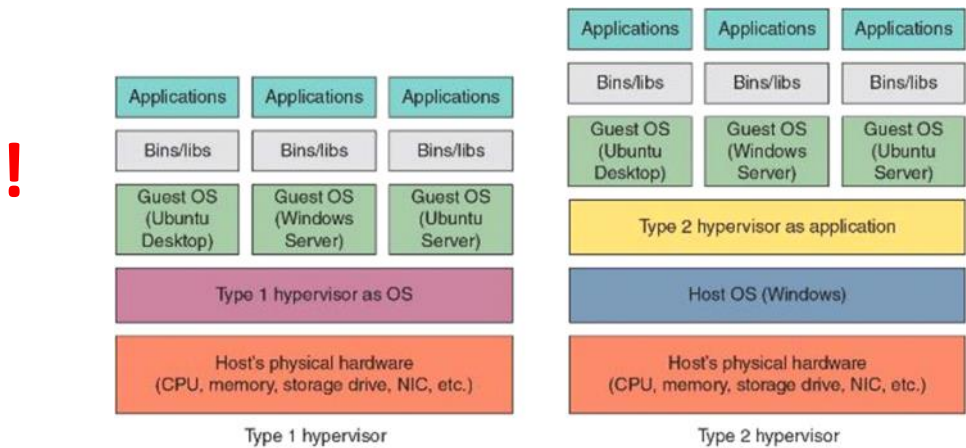


Figure 2-3 A type 1 hypervisor is installed directly on the host's firmware, while a type 2 hypervisor is installed as an application in an existing operating system

- A **Type 1 hypervisor** runs **directly on the computer's hardware**. It does not need another operating system. It manages all virtual machines itself.
- A **Type 2 hypervisor** runs **on top of an existing operating system** (like Windows). It works as an **application** that creates and controls virtual machines.

👉 Type 1 is **faster** and used in servers, while Type 2 is **easier for personal or small setups**.

Network Connection Types!!

Virtual machines use **vNICs (virtual network cards)** to connect to a network.

- The number of vNICs a VM can have depends on the **hypervisor's limits**.
- A **vSwitch (virtual switch)** connects these vNICs to the network, and one host can have **many virtual switches**.
- There are three main network modes: **Bridged**, **NAT**, and **Host-only**.

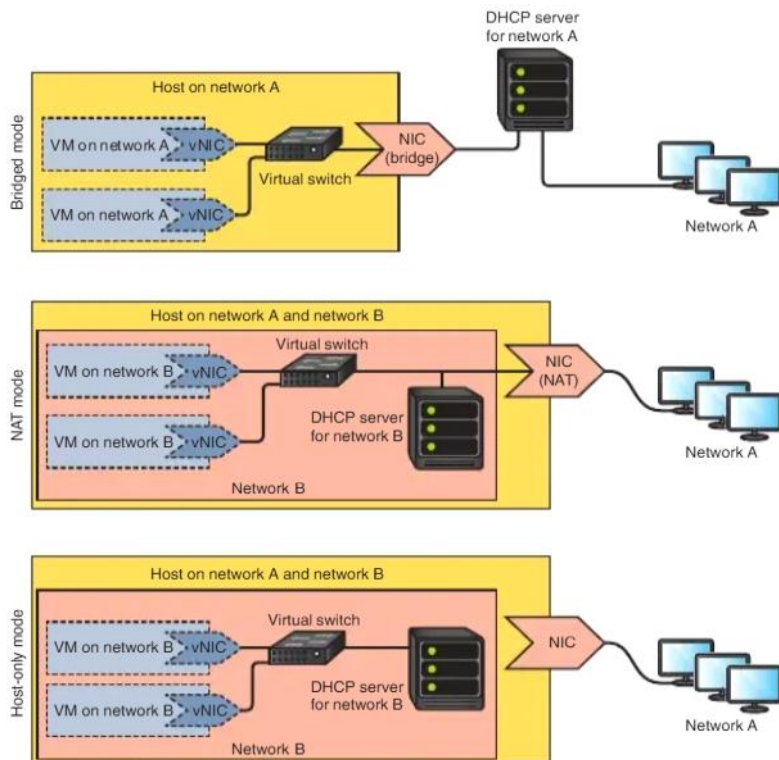


Figure 2-6 The **network mode determines** which network the VM is part of and which devices (physical or virtual) it can reach

VM Configuration !!

A **vCPU (virtual CPU)** is a part of a physical CPU that is given to a virtual machine.

- The **hypervisor** manages how VMs share the physical CPU (on the host machine) using a process called **CPU scheduling**.
- When creating a new VM, it's best to **start with one vCPU** first.

A **GPU (graphics processing unit)** helps with tasks that need a lot of visual or parallel processing.

- You can give GPU power to VMs in two ways:
 1. **Pass-through** – one VM uses the GPU directly.
 2. **Virtual (shared)** – multiple VMs share the GPU.

Memory:

- **vRAM (virtual RAM)** is part of the host's memory given to a virtual machine.
- The host must have **enough physical memory**.
- Each VM should have enough memory to run its **operating system and apps** smoothly.

Storage:

- Most hypervisors allow **flexible (dynamic) storage** for VMs.
- Problems happen when storage space is almost full.
- To fix this, you can **add more physical disks** or **use overcommit storage** to make more space available.

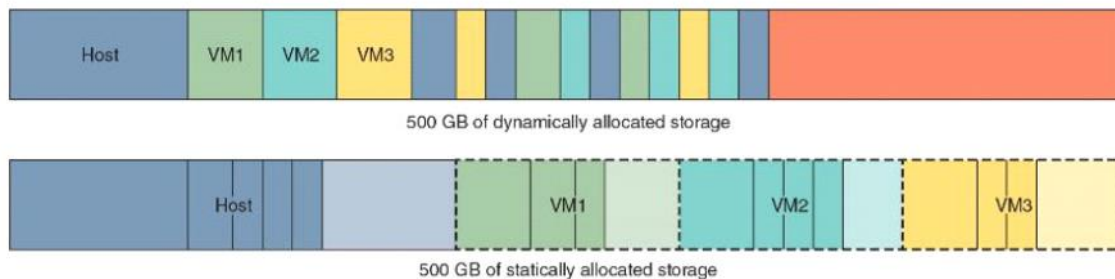


Figure 2-7 Statically allocated storage space is accessed more efficiently

It has two types of storage for virtual machines:

- **Dynamic storage:** The storage grows as the VM needs more space. It saves disk space but can be **slower** to access.
- **Static storage:** The full space (for example, 500 GB) is given to the VM from the start. It uses more space but is **faster and more efficient** to access.

👉 The best performance comes from **statically allocated storage** because it gives direct and faster access to data.

-- Device Drivers--

Virtual machines (VMs) can use **external devices** (like printers or USB drives). However, each VM must have **its own driver** to make the device work properly.

There are **three types of device drivers**:

- **Generic** – works with many devices but may have limited features.
- **Vendor** – made by the device's manufacturer, usually best performance.
- **Open source** – free drivers created by the community.

CPU Management



- **Single core (before 2002):** One core can handle **only one task at a time**. Each task it runs is called a **thread**.
- **Logical core (2002):** With **hyperthreading (HT)**, one physical core can do **two tasks at the same time**, acting like two logical cores.
- **Multicore (2005+):** CPUs started having **more than one core** (e.g., dual-core by AMD in 2005, quad-core by Intel in 2007), allowing more tasks to run in parallel for better performance.

Oversubscription (or overcommitment) means assigning more **vCPUs** to virtual machines than the number of physical CPU threads, because most VMs don't use their full CPU power at the same time.

- The ratio of **vCPUs to physical threads** is called the **overcommitment ratio**.
- The **sweet spot** means finding the best balance for this ratio depending on the type of tasks in the VMs.
- A common ratio is **3:1**, meaning three vCPUs share one physical CPU thread.

Optimizing CPU Allocation

- **SMT (Simultaneous Multithreading)** increases the number of vCPUs a host can handle, but sometimes **disabling SMT** can improve performance.
- When multiple tasks share the same physical resources, it can cause a **bottleneck**.
- **Volume-based licensing** is another reason to limit vCPU allocation, since software costs depend on the number of **logical CPU cores** being used.

Memory Management

shortage = yetersizlik



- **Memory shortage** can be managed using several memory management methods.
 - One common method is **memory paging**.
- When **RAM capacity** is full, data is stored as fixed-size files called **pages** on the hard disk, known as **virtual memory**.
 - **Virtual memory** works slower than **physical memory**.
- **Virtual memory** helps handle high memory demand periods.
 - It is also useful when assigning memory to **VMs** on a host.



Overcommitment Ratio

- Most **type 1 hypervisors** let you set RAM options for each **VM**, including:
 - **Minimum memory (reservation)**: the least amount of memory needed to start the VM
 - **Maximum memory (limit)**: the highest amount of physical memory available to the VM
 - **Startup memory**: temporary memory given only when the VM first boots
- **Dynamic memory** lets the system overcommit the host's physical memory.
- **Memory bursting** allows a VM to **temporarily use more physical memory** up to a maximum limit.

Memory Reclamation



- **Ballooning** is a method that makes the guest OS release some of its RAM so the **hypervisor** can reassign it elsewhere.
- It lets the hypervisor **reclaim extra memory** from one VM and give it to other active VMs.
- The hypervisor installs a **balloon driver** inside each guest OS to manage this process.
 - When a **high-priority VM** needs more memory, the hypervisor takes it from a **lower-priority VM** that has more than its minimum RAM, starting a **memory reclamation** process.

VM Instance Types

- In the **cloud**, you can create **VM instances** using different cloud services.
- When creating a VM instance, you can choose features like **number of vCPUs**, **memory size**, **disk type and size**, **network options**, and more.
- Cloud providers like **AWS**, **Azure**, and **GCP** offer **preconfigured instance types** where **CPU and memory** are automatically assigned in a **fixed ratio**.
- When launching multiple instances with the same configuration, you can use a **template** to make sure all are set up the same way.
- In **AWS**, an **EC2 template** includes:
 - **AMI (Amazon Machine Image) ID**
 - **Instance type**
 - **Network and security settings**
 - **Storage volumes**
- You can also include **extra parameters** within the launch template.

“Affinity” sanal makinelerin (VM) **hangi fiziksel sunucuda** çalışacağını belirleme yöntemidir. Yani, birden fazla VM’in **aynı sunucuda mı**, yoksa **farklı sunucularda mı** çalışmasını istediğini belirliyorsun.

!! Affinity

- **CAB (Cluster Across Boxes)** means several VMs are spread across multiple physical hosts in a LAN.
- When creating these clustered hosts, you can choose if VMs stay on the **same host** or on **different hosts**:
 - VMs on the same host form an **affinity group**.
 - You set this with an **affinity rule** on the VM instances.
 - If you don’t want VMs to share the same host, use an **anti-affinity rule**.
- Public cloud platforms also let you control VM placement, often by **choosing a geographical location** for the instance.

• Affinity Rule:

This rule keeps some virtual machines (VMs) on the **same physical host**.

◆ Example: If two applications always need to communicate, it’s better to keep them on the same host for better performance.

• Anti-Affinity Rule:

This rule places VMs on **different physical hosts**.

◆ Example: If one host fails, the other VM will not be affected because it runs on another host.

- Most **Cloud Service Providers (CSPs)** try to **spread** virtual machines (VMs) across different physical hosts automatically.
- If you want to control where your VMs are placed, you can use **placement groups** — these are groups of VMs that run in the same **Availability Zone (AZ)**.
- **AWS** has **three types of placement groups**:
 - **Cluster** – VMs are placed **close together** for high performance.
 - **Spread** – VMs are placed on **different hosts** for better **fault tolerance**.
 - **Partition** – VMs are divided into **separate groups** to limit impact from hardware failure.

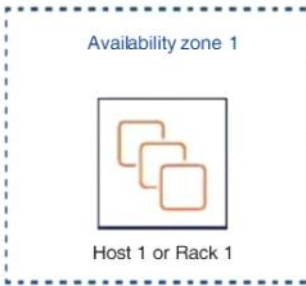


Figure 2-13 Cluster placement group

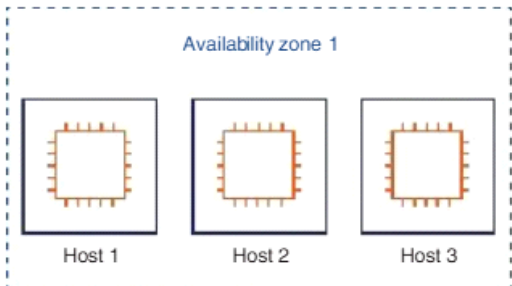


Figure 2-14 Spread placement group

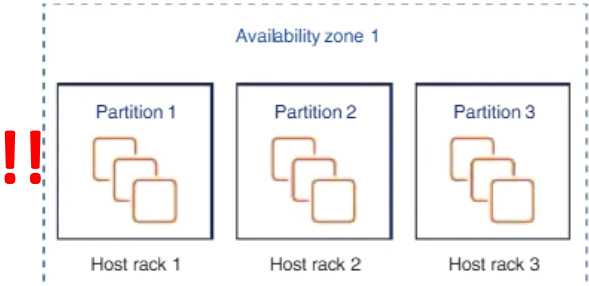


Figure 2-15 Partition placement group

Very high availability (Spread or Partition)

low-latency communication

Allocation Factors



- **Security:** Cloud systems can make security more complex.
- **High availability:** The system or network stays online and available most of the time.
- **Disaster recovery:** Virtual systems make it easier to recover from problems or data loss.
- **Energy savings:** Virtualization and cloud use less power than physical servers.
- **Cost considerations:** VM prices depend on the number and type of vCPUs.

VM Alternatives

This part talks about other ways to host apps and services instead of using Virtual Machines (VMs).

Serverless Computing (FaaS)

- In serverless computing, the **Cloud Service Provider (CSP)** gives you a server only when your app needs to run.
- This helps **save money** and lets the CSP handle all **server management**.
- It's also called **FaaS (Function as a Service)**.
- Examples: **AWS Lambda**, **Azure** and **Google Cloud Functions(GCF)**.
- **Serverless Computing** is perfect for apps or tasks that **don't need to run all the time**, like backups or data transfers.

Containers



- A **container** is a small, independent environment that includes everything needed to run an app on almost any operating system.
- The OS layer is run by a software platform called a **container engine**.
- Many containers can run on **different machines** to make systems faster and more reliable.
- Each app has its own container, which makes development, testing, and deployment easier and more stable.

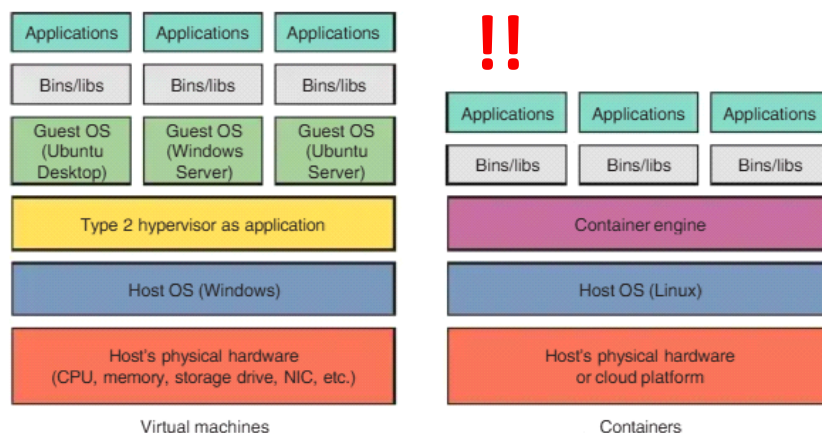


Figure 2-17 A container engine creates a lightweight, self-contained environment that provides only the files and services needed to run an application

- **Virtual machines (VMs)** need a full **guest OS** for each app, so they use **more resources**.
- **Containers** share the **same host OS** and use a **container engine** instead, making them **lighter, faster, and more efficient**.
- The container engine only provides what's needed to run the app — nothing extra

Supporting Containers

- Containers need a **containers management service** to control and organize them.
- To run containers, you also **need hosts** (servers) that support the management tool.
- **AWS ECS (Elastic Container Service)** helps to deploy, schedule, scale, and manage containers using **EC2 instances**.
- When using ECS, the user must **(responsible)**:
 - Create and connect EC2 instances to the ECS cluster.
 - Monitor and adjust resource usage.
 - Manage each host's OS, including **security and updates**.

Troubleshoot Applications in Containers

This slide explains how to fix (troubleshoot) problems in containerized applications 

- You can set **hard limits** on how much CPU or memory a container can use.
 - If no limits are set, the **container management system** will decide how to share CPU and memory between containers.
 - Adding limits **inside the container image** helps prevent overuse of host resources.
 - Containers can face **service overload** (too many requests).
- Two main solutions for overload:
1. **Rate limiting** → controls how many requests are allowed in a period.
 2. **Concurrency control** → controls how many requests can run at the same time.

Simple meaning: You can fix performance problems by limiting how much work a container can do and how fast it handles requests.